BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 03a — Les processus

```
bash — R3.05
$ ./famille
Père : pid 978, mon fils est 979
Fils : pid 979, mon père est 978
$ pstree -p $$
bash(2024)──famille(978)──famille(979)
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Programme et processus

Ce qu'est un processus, son espace mémoire, les modes utilisateur et noyau, l'arbre des processus.

02

Caractéristiques et états

PID, PPID, propriétaire… La commande ps, les états, les transitions, les signaux.

03

Créer des processus

fork, exit, wait / waitpid, zombies et orphelins, la famille exec, le mini-shell.

Suite au CM 03b : les tubes, les redirections (dup, dup2), les tubes nommés, puis la concurrence entre processus et l'exclusion mutuelle.

01

Programme et processus

Un programme est une recette, un processus est sa préparation

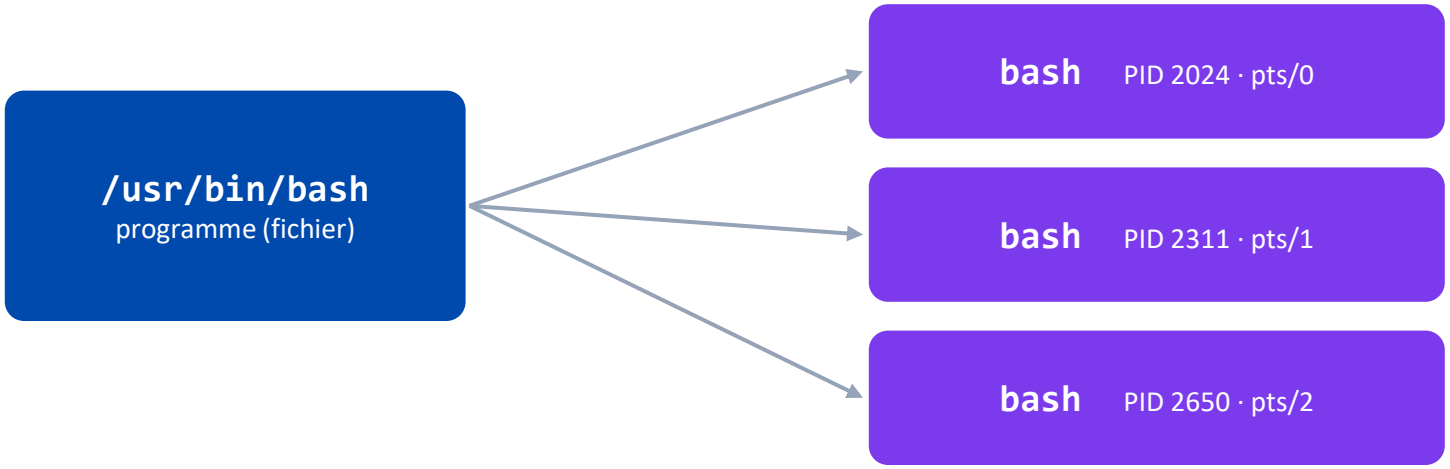
Programme ou processus ?

Un programme

Une suite d'instructions que le système fait exécuter au processeur. C'est un fichier exécutable sur le disque : il est passif.

Un processus

L'exécution d'un programme par le système, dans un environnement particulier (mémoire, fichiers ouverts, utilisateur). Il est actif.



Trois terminaux ouverts : un seul programme, trois processus, chacun avec son PID et ses données.

Observer les processus : ps et top

ps -ef : tous les processus

```
$ ps -ef | head -8
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 08:01 ?        00:00:02 /sbin/init
root           2     0  0 08:01 ?        00:00:00 [kthreadd]
root         612     1  0 08:01 ?        00:00:00 /usr/sbin/sshd -D
cruz        2011     1  0 08:12 ?        00:00:00 /lib/systemd/systemd --user
cruz        2024  2019  0 08:12 pts/0    00:00:00 -bash
cruz        3121  2024  0 09:30 pts/0    00:00:00 ps -ef
```

ps

Une photo des processus à un instant donné. ps -ef : tous, en format complet.

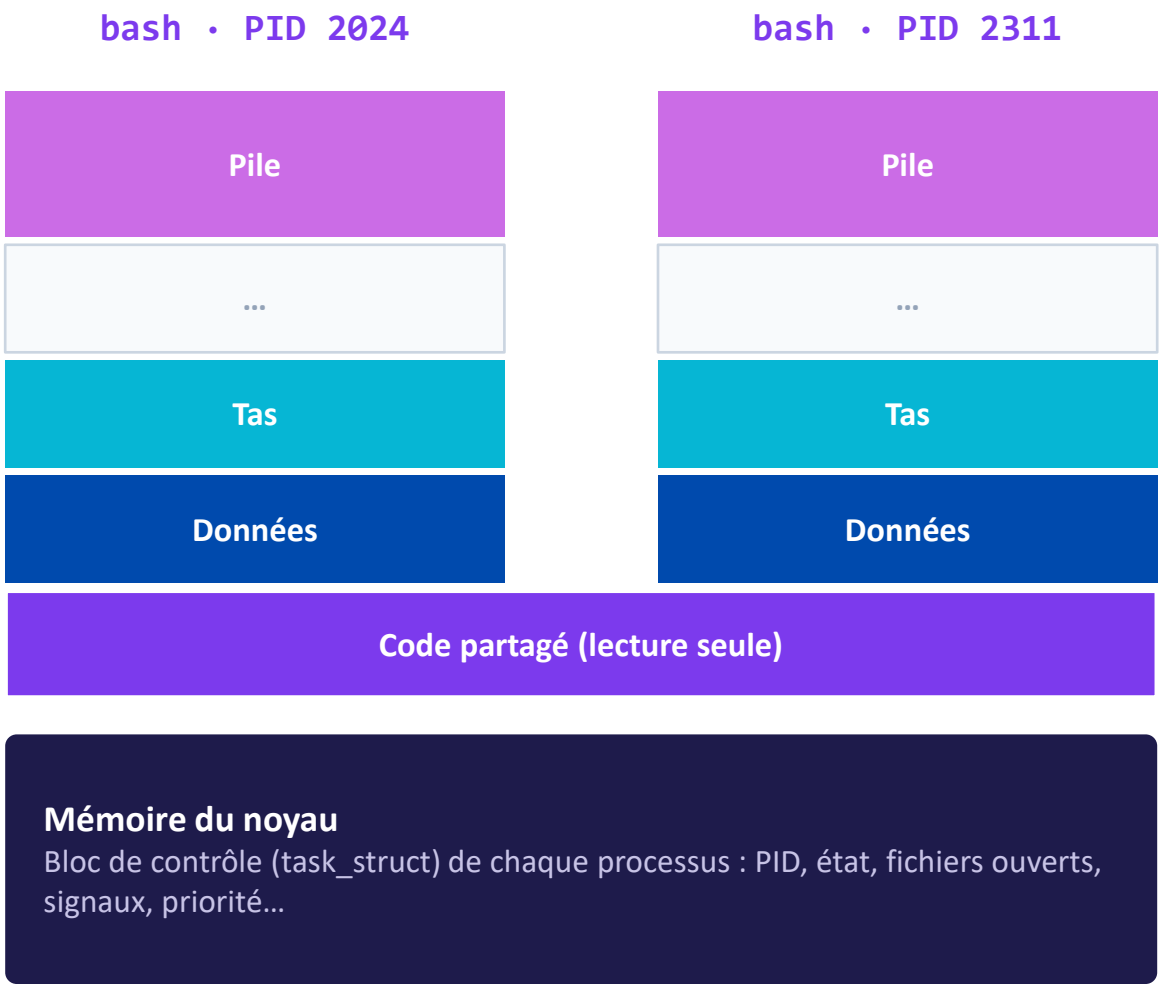
top / htop

Une vue en temps réel : CPU, mémoire, et possibilité de tuer un processus.

pstree

L'arbre de filiation des processus (pstree -p affiche aussi les PID).

Chaque processus a son espace mémoire



- Chaque processus a son propre espace d'adressage : ses données, son tas, sa pile
- Un processus ne peut pas lire ni écrire dans la mémoire d'un autre
- Plusieurs processus du même programme peuvent partager le code : il n'est jamais modifié
- Les informations utiles au système sont dans le noyau, hors de portée du processus

La table des processus du noyau

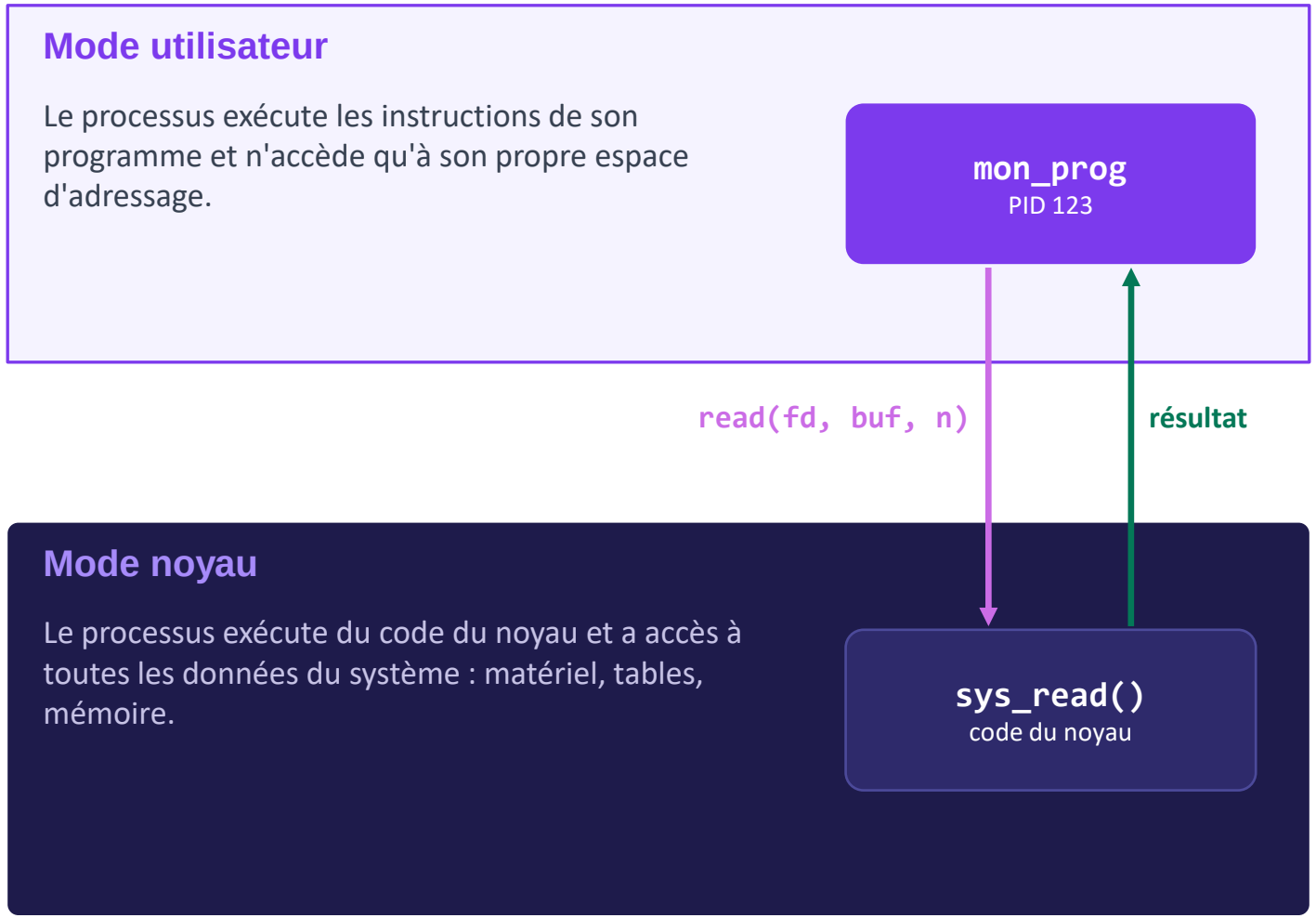
Le noyau tient une liste (doublement chaînée) de descripteurs de processus. Sous Linux, chaque descripteur est une structure task_struct.



struct task_struct (extraits)

- **pid, tgid** identifiants
- **parent, children** liens de filiation
- **files** table des descripteurs
- **prio, policy** priorité, ordonnancement
- **__state** état (prêt, endormi…)
- **mm** espace mémoire
- **cred** utilisateur, groupe
- **signal** signaux reçus

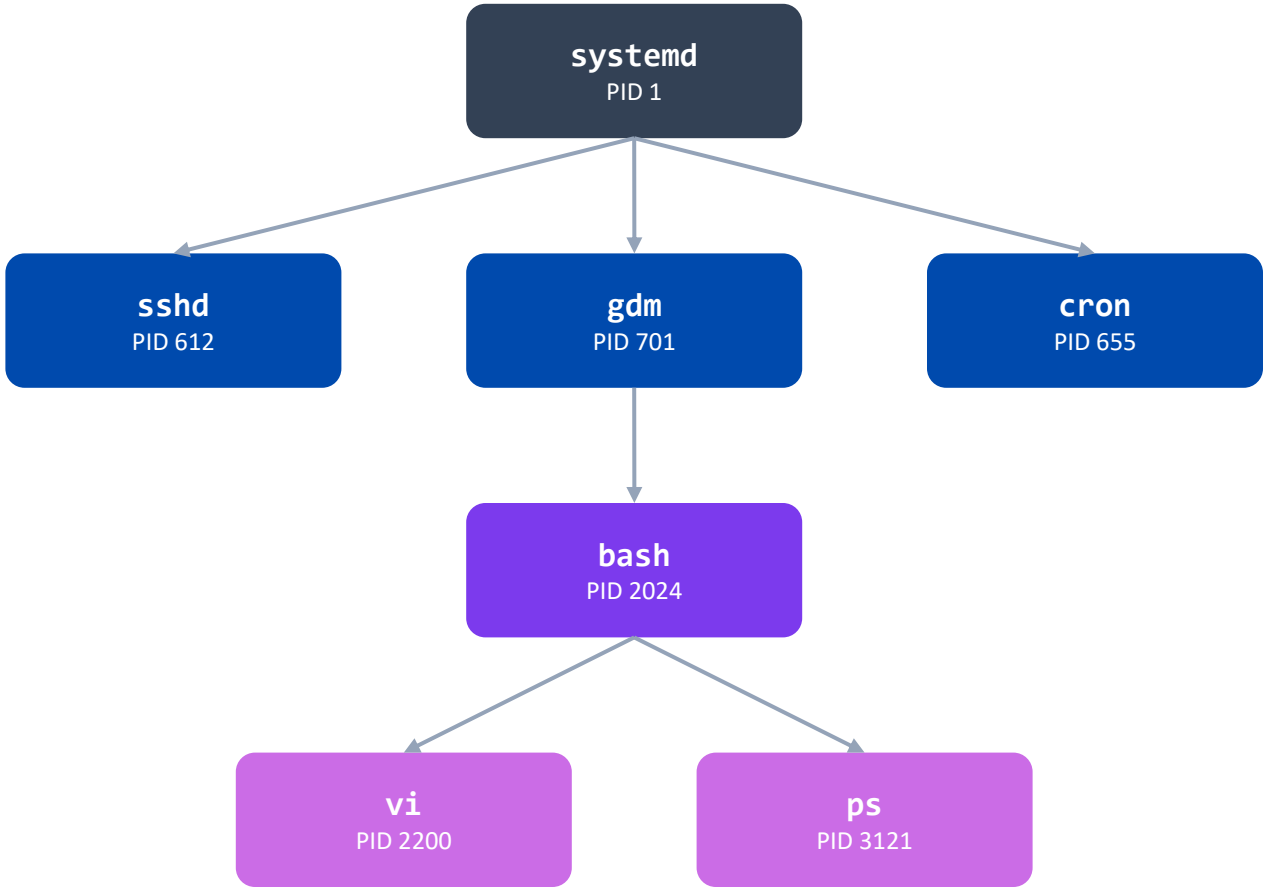
Mode utilisateur et mode noyau



- On passe en mode noyau lors d'un appel système (open, read, write, fork…) ou d'une interruption
- Le processeur interdit au mode utilisateur les instructions dangereuses
- malloc ou printf ne sont pas des appels système : ce sont des fonctions de la libc… qui en font (brk, mmap, write)

L'arbre des processus

- Tout processus est créé par un autre : son père
- Le fils hérite de l'environnement du père : variables d'environnement, répertoire de travail, fichiers ouverts…
- Le PID 0 est créé par le noyau au démarrage ; il crée init (aujourd'hui systemd), le PID 1, ancêtre de tous les processus




```
$ pstree -p 2024
bash(2024)─┬─ps(3121)
           └─vi(2200)
```

02

Caractéristiques et états

Ce que le noyau sait d'un processus

Les caractéristiques d'un processus

PID	son identifiant (type pid_t)	**PPID**	l'identifiant de son père
UID / GID	son propriétaire et son groupe : ils fixent ses droits	**Répertoire de travail**	point de départ des chemins relatifs
umask	masque de création des fichiers	**Descripteurs**	sa table des fichiers ouverts
Temps CPU	consommés en mode utilisateur et noyau	**État**	prêt, actif, endormi, arrêté, zombie
Priorité	valeur nice, de -20 à 19	**Groupe et session**	pour la gestion des tâches du shell

Un processus privilégié (root) peut changer de propriétaire : c'est ce que fait le programme de connexion pour vous donner votre identité.

Lire la sortie de ps -l

```
$ ps -l
F S   UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY        TIME CMD
0 S  1000  2024  2019  0  80   0 -  2234 do_wai pts/0  00:00:00 bash
0 T  1000  2200  2024  0  80   0 -  4521 do_sig pts/0  00:00:00 vi
4 R  1000  3121  2024  0  80   0 -  2635 -      pts/0  00:00:00 ps
```

Colonne	Signification
S	état : R (prêt ou actif), S (endormi), T (arrêté), Z (zombie)…
UID	propriétaire (0 = root, voir /etc/passwd)
PID / PPID	identifiant du processus et de son père
PRI / NI	priorité et valeur nice
SZ	taille en mémoire (en pages)
WCHAN	fonction du noyau dans laquelle il dort, s'il est endormi

Ici
bash (2024) attend la fin de ses fils (do_wait).
vi (2200) a été arrêté par Ctrl+Z (T).
ps (3121) est en cours d'exécution (R).

Les états d'un processus

R — Prêt ou actif

Prêt à s'exécuter, ou en cours d'exécution (mode utilisateur ou noyau).

S — Endormi

En attente d'un événement : une E/S, un signal, la fin d'un fils. Réveillable par un signal.

D — Endormi profond

En attente non interruptible, typiquement une E/S disque.

T — Arrêté

Suspendu par un signal (SIGSTOP, ou SIGTSTP avec Ctrl+Z). Reprend avec SIGCONT.

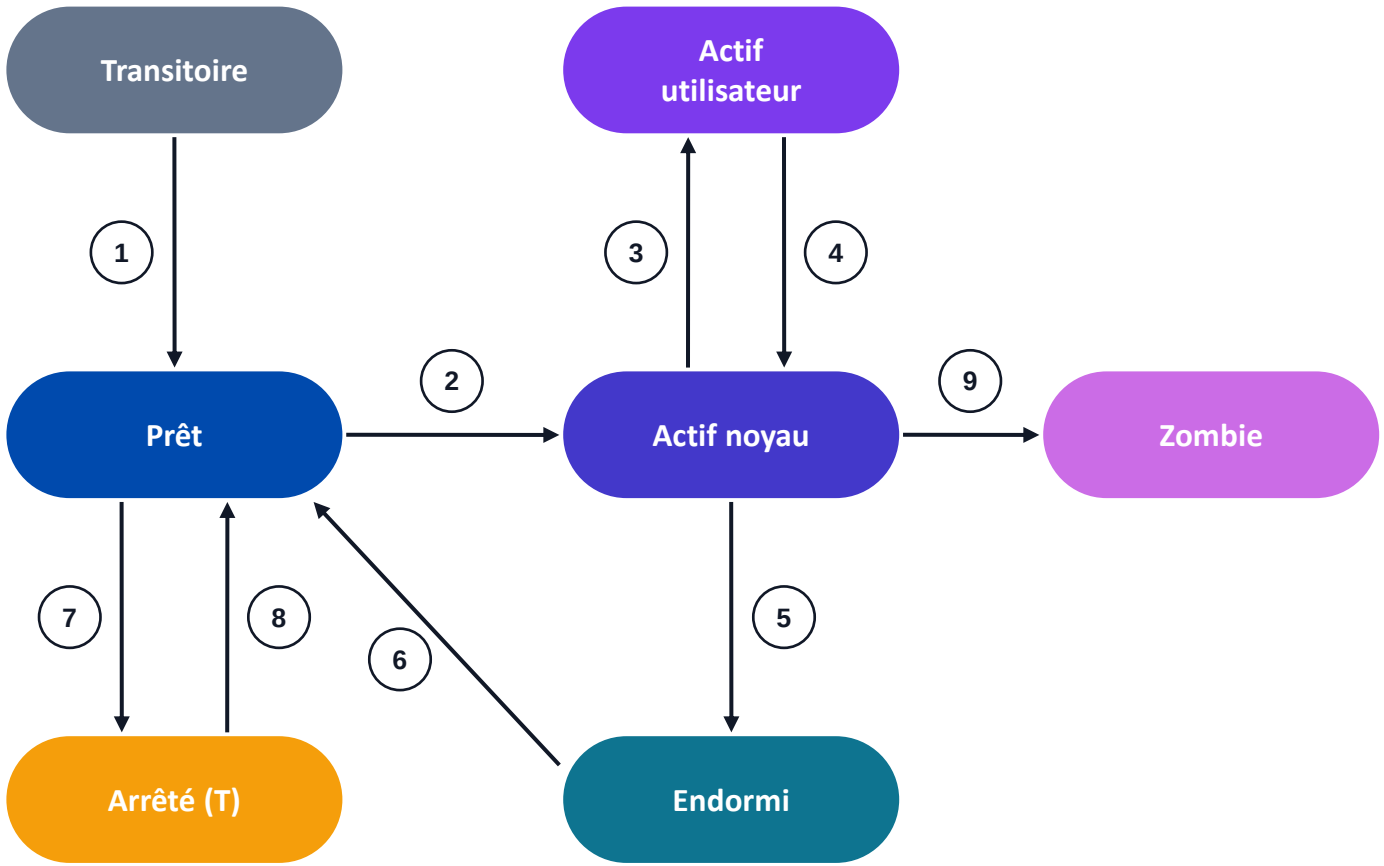
Z — Zombie

Terminé, mais son père n'a pas encore lu son code de retour (wait).

... — Transitoire

En cours de création, juste après le fork.

Les transitions entre états

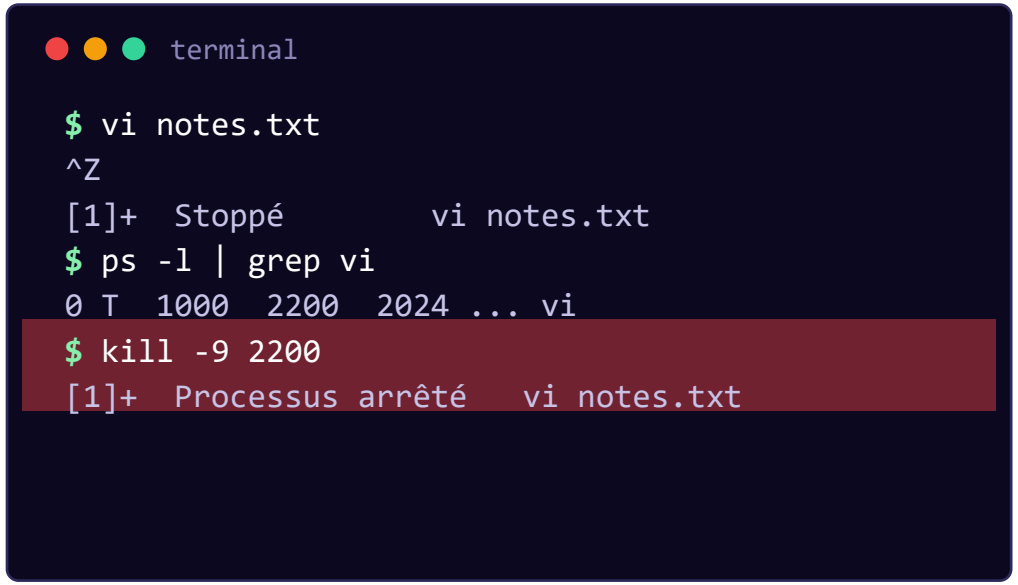


1. il a obtenu les ressources nécessaires
2. élu par l'ordonnanceur (changement de contexte)
3. retour d'un appel système ou d'une interruption
4. appel système ou interruption
5. attente d'un événement (E/S, ressource, fils)
6. l'événement attendu s'est produit
7. signal SIGSTOP ou SIGTSTP (Ctrl+Z)
8. signal SIGCONT (fg, bg)
9. le processus se termine (exit)

Agir sur un processus : les signaux

Un signal est une notification envoyée à un processus par le noyau, par un autre processus, ou par le clavier.

Signal	N°	Origine et effet par défaut
SIGINT	2	Ctrl+C : terminer
SIGKILL	9	kill -9 : terminer, impossible à intercepter
SIGTERM	15	kill (par défaut) : terminer proprement
SIGTSTP	20	Ctrl+Z : arrêter (état T)
SIGCONT	18	fg ou bg : reprendre
SIGCHLD	17	envoyé au père quand un fils se termine



En C : kill(pid, SIGTERM); envoie un signal. Les signaux seront détaillés en TP.

03

Créer des processus

fork, exit, wait, exec

Les primitives sur les processus

Identifiants

`getpid(), getppid()`

Propriétaires

`getuid(), getgid(), setuid(), setgid()`

Répertoire de travail

`getcwd(), chdir()`

Temps et priorité

`time(), nice()`

Création

`fork(), exec…()`

Fin et attente

`exit(), wait(), waitpid()`

```
#include <unistd.h>      // getpid, fork, exec, chdir…
#include <sys/wait.h>    // wait, waitpid
```

Qui suis-je ? getpid, getppid, chdir

ident.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char rep[256];
    printf("pid = %d, ppid = %d\n", getpid(), getppid());
    chdir("/tmp");
    printf("répertoire : %s\n", getcwd(rep, sizeof rep));
    return 0;
}
```

pid_t est un type entier signé, défini dans <sys/types.h> (int sous Linux).

terminal

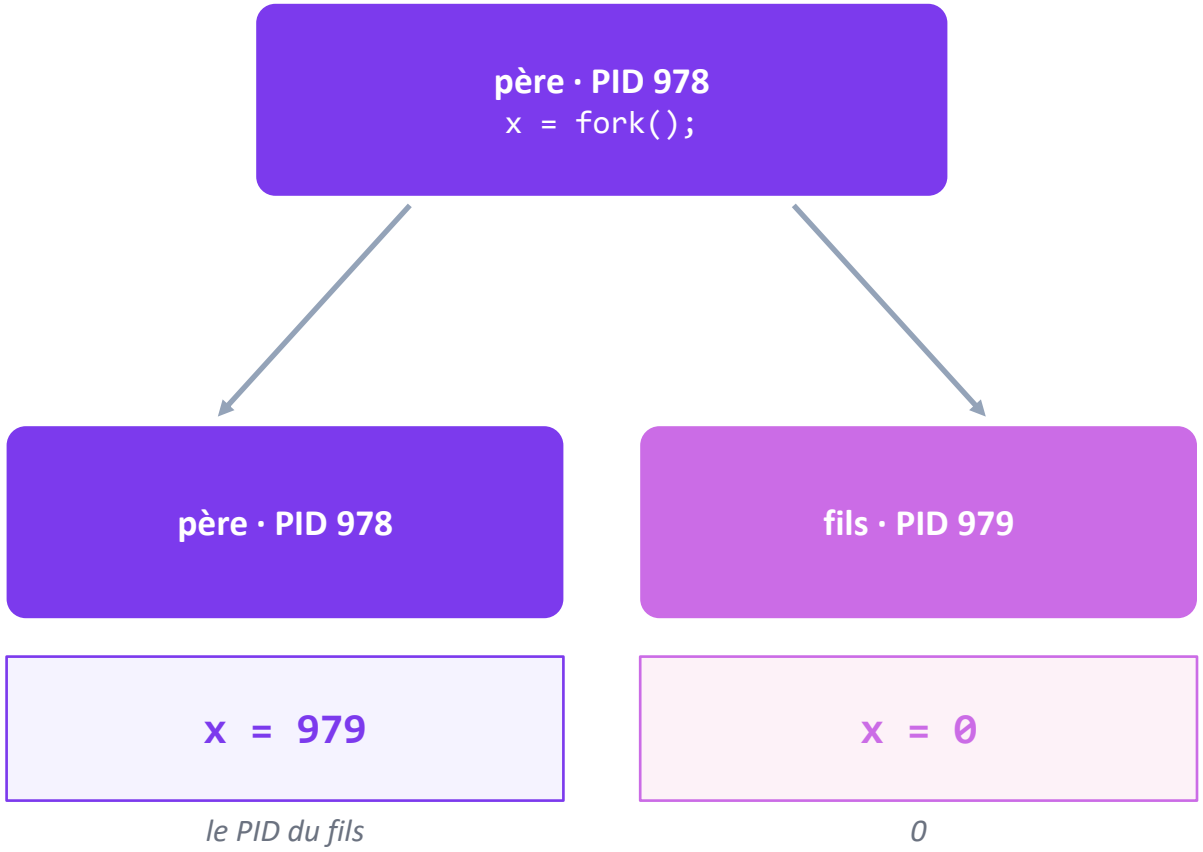
```
$ ./ident
pid = 3121, ppid = 2024
répertoire : /tmp
$ echo $$
2024
$ pwd
/home/cruz
```

chdir ne change que le processus : le shell, son père, reste dans /home/cruz. C'est pour ça que cd est une commande interne du shell, pas un programme.

Dupliquer un processus : fork

```
pid_t fork(void);
```

- Crée un nouveau processus qui s'exécute en même temps que celui qui l'a créé
- Le fils est une copie du père : code, données, pile, descripteurs ouverts…
- Seuls diffèrent le PID, le PPID… et la valeur renvoyée par fork
- Tous les processus (sauf le PID 0) sont créés ainsi



En cas d'échec, fork renvoie -1 et aucun fils n'est créé.

fork pas à pas (1)

père · PID 978

```
int a, numero;
a = 10;
numero = fork();
a = a + numero;
printf("%d\n", a);
```

a	numero
1009	999

fork

fils · PID 999

```
int a, numero;
a = 10;
numero = fork();
a = a + numero;
printf("%d\n", a);
```

a	numero
10	0

```
$ ./exemple
1009
10
```

L'ordre des affichages n'est pas garanti : c'est l'ordonnanceur qui décide.

fork pas à pas (2) : distinguer père et fils

père · PID 978

```c
int a = 0;
pid_t pid = fork();
if (pid != 0) {
    a += 10;      // père
} else {
    a += 100;     // fils
}
printf("%d\n", a);
```

a	pid
10	10078

fils · PID 10078

```c
int a = 0;
pid_t pid = fork();
if (pid != 0) {
    a += 10;      // père
} else {
    a += 100;     // fils
}
printf("%d\n", a);
```

a	pid
100	0

```
$ ./exemple2
10
100
```

Deux variables a distinctes : modifier celle du fils ne change pas celle du père.

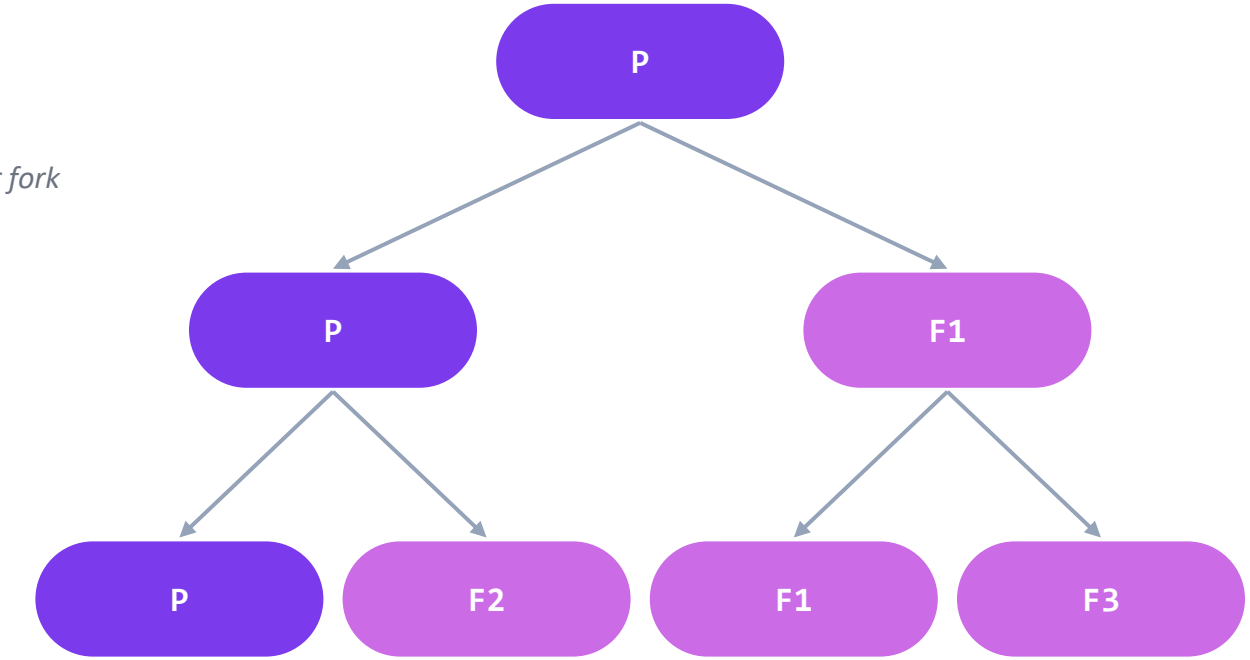
Exercice : combien de bonjour ?

combien.c

```c
int main(void)
{
    fork();
    fork();
    printf("bonjour\n");
    return 0;
}
```

2e fork

1er fork



Règle : n appels à fork exécutés à la suite donnent 2^n processus. Attention aux fork dans une boucle !

```
$ ./combien
bonjour
bonjour
bonjour
bonjour
```

Terminer et attendre : exit, wait, waitpid

```c
#include <stdlib.h>
#include <sys/wait.h>

void  exit(int code);
pid_t wait(int *statut);
pid_t waitpid(pid_t pid, int *statut, int options);
```

attente.c

```c
pid_t pid = fork();
if (pid == -1) { perror("fork"); exit(EXIT_FAILURE); }
if (pid == 0) {                          // fils
    printf("fils %d au travail\n", getpid());
    exit(3);
}
int statut;                              // père
waitpid(pid, &statut, 0);
if (WIFEXITED(statut))
    printf("fils fini, code %d\n", WEXITSTATUS(statut));
```




```
$ ./attente
fils 979 au travail
fils fini, code 3
```

- wait : attend la fin de n'importe quel fils
- waitpid : attend un fils précis ; WNOHANG pour ne pas bloquer
- WEXITSTATUS : récupère la valeur passée à exit (0 à 255)

Zombies et orphelins

Le zombie

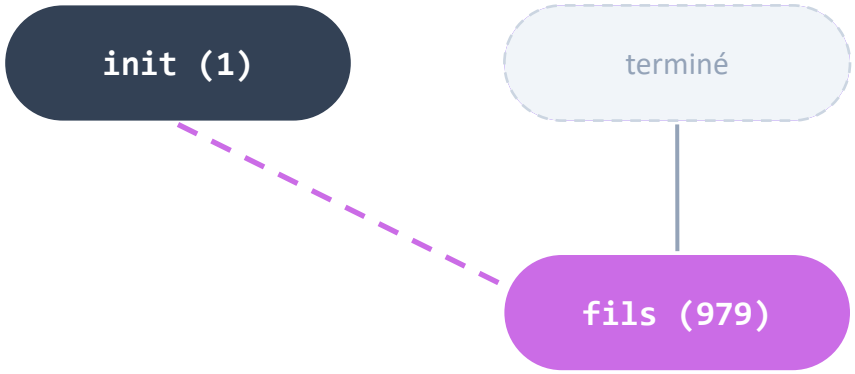
Le fils est terminé, mais son père n'a pas encore appelé wait. Le noyau garde son entrée (PID, code de retour) dans la table des processus.

```
$ ps -l
0 S 1000 978 2024 ... pere
1 Z 1000 979  978 ... pere <defunct>
```

Risque : un père qui crée beaucoup de fils sans jamais faire wait remplit la table des processus. Solution : toujours attendre ses fils.

L'orphelin

Le père se termine avant son fils. Le fils est alors adopté par init (PID 1) ou par un processus « collecteur », qui fera wait pour lui.

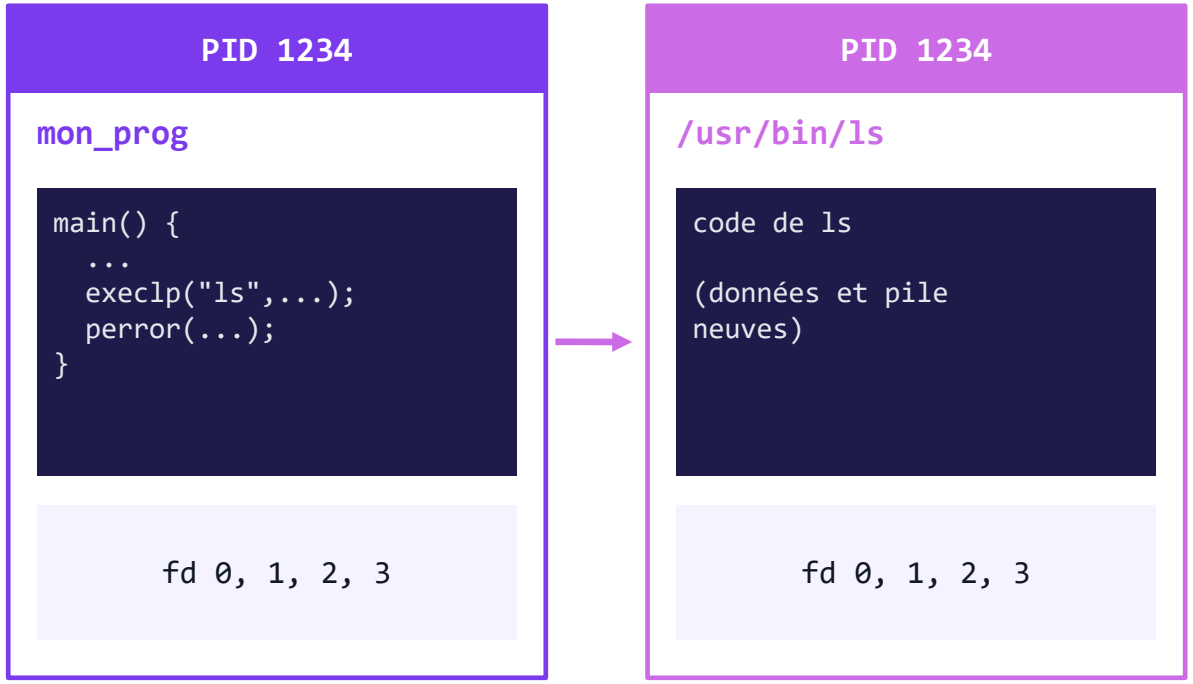


nouveau père : PPID = 1

Changer de programme : la famille exec

```
int execve(const char *chemin, char *const argv[], char *const envp[]);
```

- exec remplace le code, les données et la pile du processus par ceux d'un nouveau programme
- Le processus garde son PID, son père, ses descripteurs ouverts
- En cas de succès, exec ne revient jamais : les instructions qui suivent ne sont exécutées que si exec échoue
- argv : les arguments (argv[0] = nom du programme) ; envp : l'environnement



même PID, mêmes descripteurs

Six variantes, une logique

Fonction	Arguments	Recherche dans PATH	Environnement
execl	liste … NULL	non	hérité
execlp	liste … NULL	oui	hérité
execle	liste … NULL	non	fourni (envp)
execv	tableau argv[]	non	hérité
execvp	tableau argv[]	oui	hérité
execve	tableau argv[]	non	fourni : l'appel système

```c
execlp("ls", "ls", "-l", NULL);
char *args[] = {"ls", "-l", NULL};
execvp("ls", args);              // équivalent
```

l — **liste**
les arguments sont écrits un par un, terminés par NULL

v — **vecteur**
les arguments sont dans un tableau terminé par NULL

p — **PATH**
on donne seulement « ls », le système cherche /usr/bin/ls

e — **environnement**
on fournit soi-même les variables d'environnement

Exemple : execlp

lance_ls.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("avant exec\n");
    execlp("ls", "ls", "-l", NULL);
    perror("execlp");     // seulement si exec a échoué
    return 1;
}
```

```
$ ./lance_ls
avant exec
total 24
-rwxr-xr-x 1 cruz info 16032 lance_ls
-rw-r--r-- 1 cruz info   190 lance_ls.c
```

perror n'est jamais affiché : le programme lance_ls a été remplacé par ls.

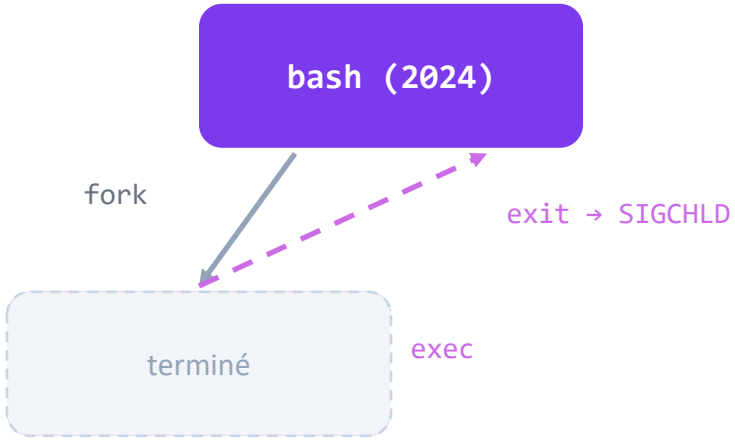
Piège des tampons : sans \n, ou si la sortie est redirigée vers un fichier, « avant exec » reste dans le tampon de printf et est perdu. Appelez fflush(stdout) avant exec.

Ancienne version du cours : deux execlp à la suite. Le second n'est jamais atteint si le premier réussit.

fork + exec + wait : ce que fait le shell

mini_shell.c

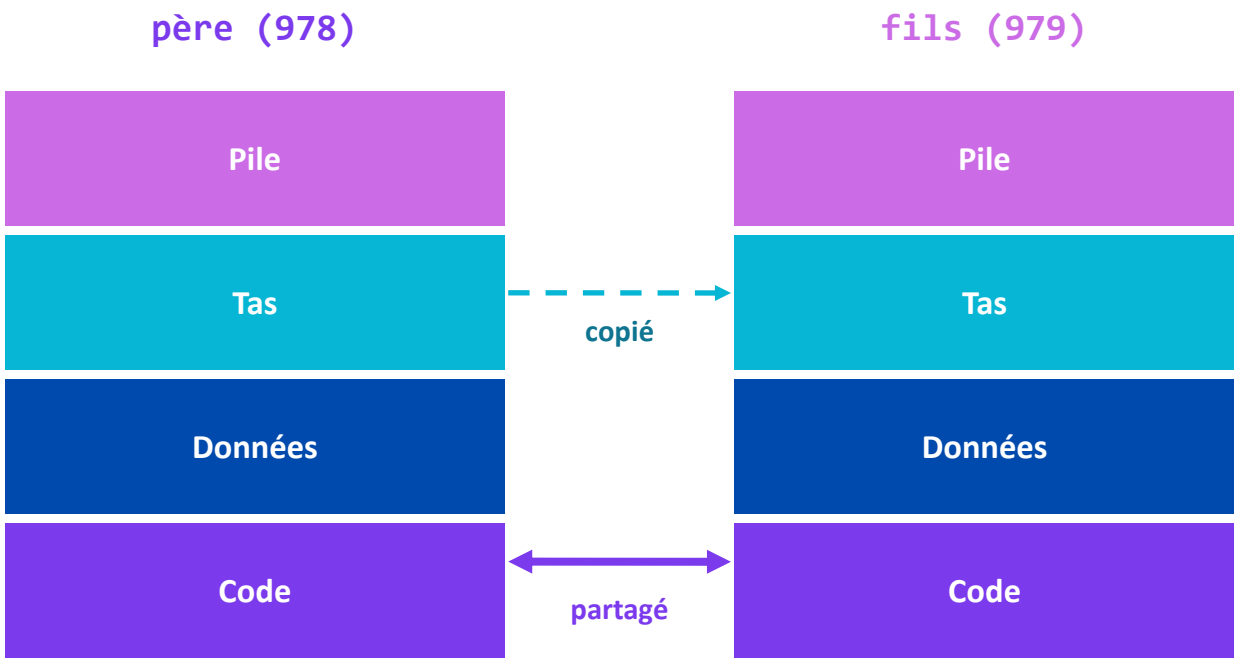
```c
pid_t pid = fork();
if (pid == 0) {                          // fils
    execlp("ls", "ls", "-l", NULL);
    perror("execlp");
    exit(127);                           // commande introuvable
}
waitpid(pid, NULL, 0);                   // père
printf("commande terminée\n");
```



waitpid revient : le shell affiche le prompt suivant

C'est exactement ce que fait votre shell à chaque commande. Avec « ls & », il ne fait pas waitpid tout de suite : la commande tourne en arrière-plan. Et 127, le code de « commande introuvable », est celui que renvoie bash.

La mémoire après un fork



Bloc de contrôle (noyau) : chacun le sien, avec son PID, son état, sa table des descripteurs (copiée lors du fork).

Le code

Il est en lecture seule et n'est jamais modifié : père et fils partagent les mêmes pages en mémoire.

Copie à l'écriture

Copier toute la mémoire serait lent. Linux partage d'abord toutes les pages, et ne copie une page que lorsque l'un des deux processus la modifie (copy-on-write).

SYNTHÈSE

À retenir

01

Processus

Un programme en exécution, avec son propre espace mémoire et un bloc de contrôle dans le noyau.

02

États

R, S, D, T, Z. On passe de l'un à l'autre par l'ordonnanceur, les E/S et les signaux.

03

fork

Duplique le processus. Renvoie 0 au fils, le PID du fils au père, -1 en cas d'erreur.

04

exit / wait

Le père doit attendre ses fils, sinon ils restent zombies. WEXITSTATUS lit le code.

05

exec

Remplace le programme, garde le PID et les descripteurs. Ne revient qu'en cas d'échec.

06

Le shell

fork, puis exec dans le fils, puis wait dans le père : à chaque commande.

Ressources pédagogiques

Le manuel

```
$ man 2 fork
$ man 2 execve
$ man 3 exec        # execl, execlp…
$ man 2 waitpid
$ man 7 signal
$ man ps
```

Explorez aussi /proc/<pid>/ : status, fd/, maps… Le noyau y montre les informations de chaque processus.

Lectures

- A. Tanenbaum, Systèmes d'exploitation, Pearson : chapitre « Processus et threads »
- J.-M. Rifflet, La programmation sous UNIX, Ediscience
- M. Kerrisk, The Linux Programming Interface, No Starch Press : chapitres 24 à 28

lamae

Questions ?

PROCHAINE SÉANCE

CM 03b — Tubes et concurrence

pipe, dup2, tubes nommés · sections critiques · exclusion mutuelle

Christophe Cruz · christophe.cruz@ube.fr